Urban Mobility Data Explorer: Technical Documentation Report

1. Problem Framing and Dataset Analysis

The Urban Mobility Data Explorer analyzes New York City Yellow Taxi trip data for January 2019. The project aims to clean and analyze transportation data and present it in a way that helps users understand taxi demand, trip patterns, fares, and service zones.

The original dataset contained 7,667,792 trips. After cleaning, we retained 7,649,688 trips (99.76%). This means that 18,104 records (0.24%) were removed during the cleaning process.

The cleaning process checked for missing values, invalid trip durations, unrealistic distances and fares, incorrect passenger counts, and pickup and drop-off location identifiers. These checks helped remove records that did not meet the data quality requirements while preserving most of the original dataset.

One challenge was the mismatch between the geographic reference files. The zone lookup contained 265 zones, while the GeoJSON file contained 260 zones, a difference of five zones. Some zone names were also duplicated, meaning that names alone could not always be used to identify locations correctly.

2. System Architecture and Design Decisions

The system is divided into three main parts: the frontend, backend, and database. Each part has a specific responsibility.

The frontend uses HTML, CSS, JavaScript, and Chart.js to display information through interactive charts and visualizations. The backend uses Python and Flask to handle requests, retrieve data, and perform analysis. The database uses CockroachDB, which is compatible with PostgreSQL, to store trip records and related information. GeoJSON supports the geographic representation of service zones.

The backend provides API endpoints for retrieving general trip statistics, trip records, transportation demand, fare information, service zones, and analytical results. The database also stores supporting information such as boroughs, taxi zones, vendors, rate codes, and payment types.

Separating these components makes the system easier to maintain. For example, changes to the analysis logic can be made in the backend without rebuilding the entire frontend.

3. Algorithmic Logic and Data Structures

The project uses several algorithms and data structures to organize and analyze the transportation data.

Merge sort — O(n log n)

Merge sort organizes records by dividing the data into smaller sections, sorting them, and combining the results. It provides predictable sorting performance, even when working with large datasets.

Quickselect — Average O(n)

Quickselect finds a particular ranked value without sorting the entire dataset. This can save time when only a specific result or top-ranked group is required.

Custom hash map — Average O(1) lookup

A hash map stores information using keys, allowing values to be retrieved quickly. In this project, it is useful for matching identifiers with information such as taxi zones.

Z-score detection — O(n)

Z-score analysis helps identify values that differ significantly from the average. The formula is:

$$z = \frac{x - \mu}{\sigma}$$

Here, x is the observed value, μ is the mean, and σ is the standard deviation. Values with large absolute Z-scores can be investigated as unusual observations, although they are not necessarily errors.

The project also derives additional features, including trip duration and fare per mile. These measures help compare trips and examine the relationship between distance and fares.

4. Insights and Interpretation

Insight 1: Most records passed the cleaning process.

Of the original 7,667,792 trips, 7,649,688 were retained, representing 99.76% of the dataset. Only 18,104 records, or approximately 0.24%, were removed. This shows that most records met the applied cleaning rules, although retention does not guarantee that every detail is accurate.

Insight 2: Geographic data contains matching problems.

The zone lookup contained 265 zones, compared with 260 in the GeoJSON file. The five-zone difference and the presence of duplicate names show why reliable identifiers are important when matching locations. Incorrect matches could affect geographic analysis and visualizations.

Insight 3: Fare per mile can decrease as trip distance increases.

The analysis indicates that longer trips can have a lower fare per mile than shorter trips. This suggests that fares do not always increase at the same rate as distance. Trip duration, traffic conditions, and fare rules may also affect the final cost.

These findings show how data cleaning, geographic validation, and derived measures can help make transportation data more useful.

5. Reflection and Future Work

One of the main challenges was processing millions of trip records while maintaining data quality and efficient analysis. Matching geographic files was another difficulty because the zone counts did not agree and some names were duplicated. Choosing appropriate algorithms and organizing the database were also important for making the system manageable.

The project could be improved by introducing additional validation rules, investigating unusual records, and correcting geographic matching problems. Database queries could be optimized to improve performance when working with large datasets.

The visualizations could also include more filtering options and clearer explanations of the results. Using data from additional months would allow comparisons across different periods and help identify seasonal changes in taxi demand.

Overall, the Urban Mobility Data Explorer combines data cleaning, algorithms, database design, and visualization to help users understand taxi trip patterns. Although some data quality and geographic matching issues remain, the project provides a useful starting point for further analysis of urban transportation.